

Core Docker Concepts #Docker #Architecture

#Containers

VM vs Container

VM (Full House): Complete operating system. Heavy, slow to start, takes gigabytes.

Container (Apartment): Shared building infrastructure (host OS kernel), isolated space. Starts in seconds, weighs megabytes.

- **VM:** [Your App] + [Libraries] + [Guest OS] → Hypervisor → Host OS
- **Container:** [Your App] + [Libraries] → Docker Engine → Host OS

Image vs Container

Image: A recipe.

Container: The meal made from that recipe. One image can produce multiple running containers simultaneously.

Image → docker run → Container (running instance)

Part 1 — Basic Commands #CLI #Commands

```
Shell
Bash
```

```
Shell

# Test installation
docker run hello-world

# Pull images without running
docker pull nginx
docker pull ubuntu
docker pull python:3.11-slim

# See all local images
docker images

# Run nginx in background (-d) and map port 80 (-p)
docker run -d -p 80:80 --name my-nginx nginx

# View containers
docker ps          # Running containers
docker ps -a       # ALL containers (including stopped)
```

```

# Enter a running container (like SSH)
docker exec -it my-nginx bash
exit

# Stop and remove
docker stop my-nginx
docker rm my-nginx           # Must be stopped first
docker rm -f my-nginx        # Force remove while running
docker rmi nginx             # Remove image
docker system prune          # Nuclear option: remove everything
unused

```

Note on -p 8000:8000: Left side = your machine port. Right side = container port.

Docker pull mysql

Docker run -d --name c1 -e MYSQL_ROOT_PASSWORD=123 -p 3306:3306 mysql

Docker exec -it c1 bash

Mysql -u root -p

SHOW DATABASES;

Create database test;

USE test;

```

CREATE TABLE users ( id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) ); INSERT
INTO users (name) VALUES ('Mayssen');
SELECT * FROM users;

```

Part 2 — Dockerfile & .dockerignore #Dockerfile

#Python #Config

Create a Dockerfile in your project root:

```

Shell
Dockerfile

```

```

Shell
# Base lightweight image
# → This is the starting environment for your container.

```

```
FROM python:3.11-slim

# Set working directory inside container

WORKDIR /app

# Copy requirements FIRST to leverage Docker layer caching
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the code
COPY . .

# Document port (does not open it, use -p for that)
EXPOSE 8000

# Command to run on start (--host 0.0.0.0 is critical for external
access)

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Package	Purpose
FastAPI	Framework to build APIs
uvicorn[standard]	Server that runs the FastAPI app
jinja2	Renders HTML templates
python-multipart	Handles forms and file uploads

Create a `.dockerignore` to exclude files from the image:

None

```
.dockerignore
```

None

```
__pycache__  
*.pyc  
venv/  
.venv/  
.env  
.git  
.gitignore
```

Part 3 — Build and Run #Build #Run

Shell

Bash

Shell

```
# Build the image (-t = tag name, . = current directory context)  
docker build -t onepiece-voyage .
```

```
# Verify image exists  
docker images
```

```
# Run container  
docker run -p 8000:8000 onepiece-voyage
```

```
# Run in background with a name  
docker run -d -p 8000:8000 --name onepiece onepiece-voyage
```

```
# Verify running & view logs  
docker ps
```

```
docker logs -f onepiece
```

```
# Enter container
```

```
docker exec -it onepiece bash
```

```
ls
```

```
cat requirements.txt
```

```
exit
```

```
# Clean up
```

```
docker stop onepiece
```

```
docker rm onepiece
```